>_ MCMP113 · Week 3

Programming Languages, Variables, Data Types and Constants

From an algorithm to a running C# program: storing, typing, reading and printing data

Asst. Prof. Hasan Oğuz, PhD
Introduction to Basic Algorithms and Programming · Istanbul Okan University
Vocational School · 2026–2027 Fall

How to use these notes

These notes follow the Week 3 slides and contain everything that does not fit on a slide: the full explanation of each idea, every example program with a real console run, trace tables, an English–Turkish glossary and graded exercises. This is the first week with real C#. **Type every example yourself** and run it; do not just read it. All example programs are in the folder `code/` (one project per example, e.g. `03_07_Receipt`); the listings below are printed directly from those tested files (without their two header-comment lines), and every console output shown was produced by running them with .NET 10. In console boxes, **amber text** is what the user typed.

Contents

1 Learning outcomes 2
2 Programming languages and .NET 2
3 Output 5
4 Identifiers and variables 6
5 Data types 8
6 Constants 11
7 Input and type conversion 12
8 Formatted output 18
9 Common mistakes 19
10 Summary 20
11 Exercises 21
References and further reading 22

>_ 1 Learning outcomes

After this week you should be able to:

1. explain the difference between machine code, assembly and high-level languages, and between compiling, interpreting and just-in-time (JIT) compilation;
2. describe how a C# program is turned into IL by the compiler and into machine code by the .NET runtime; distinguish the .NET SDK from the runtime;
3. create, build and run a console project with `dotnet new console` and `dotnet run`, and explain the files `Program.cs` and `.csproj`;
4. print text with `Console.Write`/`WriteLine`, escape sequences, verbatim strings, interpolation and format specifiers;
5. declare, initialise and assign variables with legal and meaningful names, and trace their values in a table;
6. choose a suitable data type (`int`, `long`, `double`, `decimal`, `char`, `string`, `bool`, ...) and use `const` for fixed values;
7. read input with `Console.ReadLine()` and convert it with `int.Parse`, `double.Parse` or `Convert`; recognise truncation, overflow and the decimal-comma trap.

>_ 2 Programming languages and .NET

2.1 Three levels of language

A processor (CPU) understands only **machine code**: binary instructions, each a few bytes long, specific to one processor family (x86/x64, ARM, ...). Writing programs as bit patterns is almost impossible for humans, so programmers invented **assembly language**, which gives each machine instruction a short name (a *mnemonic*). A program called an **assembler** translates it to machine code. **High-level languages** such as C, Java, Python and C# go much further: they look like a mixture of English and mathematics, are independent of the processor, and one line may become many machine instructions.

Level	"Store the number 7"	Characteristics
Machine code (x86)	`10111000 00000111 00000000 00000000 00000000` (hex `B8 07 00 00 00`)	What the CPU executes. Unreadable for people, tied to one CPU family.
Assembly	`mov eax, 7`	One mnemonic per machine instruction; translated by an assembler. Still CPU-specific.
High-level (C#)	`int x = 7;`	Readable, portable; translated by a compiler (or interpreter).

2.2 Compiled, interpreted, or just-in-time?

A high-level program must be translated before the CPU can execute it. There are three main strategies:

- **Compiled ahead of time** (C, C++, Go, Rust): a **compiler** translates the whole program into machine code *before* it runs. The result starts and runs fast, but must be rebuilt for every operating system and processor.

- **Interpreted** (Python, shell scripts, classic BASIC): an **interpreter** reads the program while it runs and executes it statement by statement. Convenient for experimenting, usually slower. (Real systems mix the ideas: CPython first compiles to bytecode and then interprets that bytecode.)
- **Intermediate language + JIT** (C#, VB.NET, Java): the compiler produces an *intermediate language*; when the program runs, a **just-in-time (JIT) compiler** inside the runtime translates it into machine code for the actual CPU. The same compiled file runs on Windows, Linux and macOS.

2.3 How a C# program runs

Figure 1 shows the path from your source file to the processor. `dotnet build` runs the C# compiler (named *Roslyn*); it checks your code and, if there are no errors, writes **IL** (Intermediate Language, also called CIL) into a `.dll` file. `dotnet run` builds and then starts the program: the **CLR** (Common Language Runtime) loads the `.dll` and its JIT compiler turns the IL into native machine code.



Figure 1: From source code to machine code. Compile-time errors stop the build; run-time errors (exceptions) stop a running program.

This picture explains two kinds of errors you will meet. **Compile-time errors** (codes such as `CS0165`) are found by the compiler; the program is not built at all. **Run-time errors** (*exceptions*, such as `FormatException`) happen while a correctly compiled program runs, for example when the user types letters where a number is expected.

SDK versus runtime

The **.NET runtime** contains what is needed to *run* a finished program: the CLR with its JIT compiler and the base class library (`Console`, `Math`, ...). The **.NET SDK** (Software Development Kit) contains the runtime *plus* the compiler, the `dotnet` command-line tool and project templates. Developers install the SDK; users of your program need only the runtime. In this course we use the .NET 10 SDK (a long-term-support release, November 2025) with C# 14.

2.4 Creating and running a project

Open a terminal (in VS Code: *Terminal* ▷ *New Terminal*) in a folder for the course and type:

```
dotnet --version                # shows e.g. 10.0.112 (any 10.0.x is fine)
dotnet new console -n Hello     # creates the folder Hello with a project
cd Hello
dotnet run                      # builds and runs: prints Hello, World!
```

`dotnet new console` creates `Hello/Program.cs` and `Hello/Hello.csproj` and prepares ("restores") the project. `dotnet run` must be started *inside* the project folder; otherwise you get *Couldn't find a project to run*. In VS Code with the C# Dev Kit, open the *folder* (*File* ▷ *Open Folder*), then use *Run* ▷ *Run Without Debugging* (Ctrl+F5). The build puts its output in the folders `bin/` and `obj/`; you never edit them and may delete them at any time.

2.5 The project file and our course template

The `.csproj` file is an XML file that tells the SDK how to build the program. Our course template differs from the default in two lines:

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>disable</Nullable>
    <InvariantGlobalization>true</InvariantGlobalization>
  </PropertyGroup>
</Project>
```

- `OutputType Exe`: build a runnable program; `net10.0`: target .NET 10; `ImplicitUsings`: common namespaces such as `System` are imported automatically, so we can write `Console` instead of `System.Console`.
- `Nullable disable`: the default template has `enable`. Then the compiler warns about possible `null` values, e.g. `string name = Console.ReadLine();` gives warning CS8600 and `int.Parse(Console.ReadLine())` gives CS8604. The warnings are justified in professional code, but confusing in Week 3, so we switch them off.
- `InvariantGlobalization true`: numbers are read and printed with a decimal *point* on every computer, including Turkish Windows (Section 7.3).

To use the template: create the project with `dotnet new console -n Name` and edit these two lines (or copy the template from the course group).

2.6 Top-level statements and the classic `Main` method

Since C# 9 a program can consist simply of statements written at the top of `Program.cs` — **top-level statements**. The compiler generates the surrounding class and the `Main` method for you. Many books and web pages still show the *classic* form, in which execution starts at the first statement of `static void Main()`, the **entry point**. Example 2 is such a program; it behaves exactly like a top-level program. In MCMP113 we write top-level statements.

Listing 1: `code/03_02_ClassicHello` – the classic template

```
1  namespace ClassicHello
2  {
3      class Program
4      {
5          static void Main()
6          {
7              // Execution starts here.
8              Console.WriteLine("Hello from Main!");
9              Console.WriteLine("The classic form.");
10         }
11     }
12 }
```

Output

```
Hello from Main!
The classic form.
```

2.7 Statements, semicolons, case and comments

- A **statement** is one instruction; simple statements end with a semicolon. Forgetting it gives `error CS1002: ; expected`.
- C# is **case-sensitive**: `Console` and `console` are different names (`console.WriteLine` gives CS0103 *The name 'console' does not exist in the current context*); `age` and `Age` are two different variables.
- **Comments** are notes for people and are ignored by the compiler: `//` until the end of the line, `/* …*/` for several lines.
- **Whitespace** (spaces, tabs, line breaks) between words and symbols is ignored, but indent consistently with 4 spaces so that the structure is visible. In VS Code, Shift+Alt+F formats the whole file.
- Braces `{ }` group statements into a *block* (you will use them from Week 5 on). We put each brace on its own line (Allman style, the C# default).

>_ 3 Output

3.1 `Write` and `WriteLine`

`Console.WriteLine(x)` prints `x` and then moves the cursor to the next line; `Console.Write(x)` leaves the cursor directly after the text. We therefore use `Write` for prompts, so the user types on the same line. `Console.WriteLine();` with nothing in the parentheses prints an empty line.

3.2 Escape sequences and verbatim strings

Inside a string literal the backslash starts an **escape sequence**:

Escape	Meaning	In code	Printed
`\n`	new line	`"Line 1\nLine 2"`	`Line 1` (new line) `Line 2`
`\t`	tab (next multiple of 8 columns)	`"Name:\tAyşe"`	`Name:  Ayşe`
`\"`	double quote	`"Say \"hi\""`	`Say "hi"`
`\\`	backslash	`"C:\\MCMP113"`	`C:\MCMP113`
`\'`	single quote (in a `char`)	`'\''`	`'`

A **verbatim string** starts with `@` and treats the backslash as an ordinary character: `@"C:\MCMP113"`. This is convenient for Windows paths. Without `@`, `"C:\Temp\new"` does not compile (`CS1009: Unrecognized escape sequence`, because `\T` is not an escape) — and `\n` would silently become a line break.

3.3 Example 1: a personal student card

Listing 2: `code/03_01_HelloCard` – Write vs WriteLine and escape sequences

```
Console.WriteLine("Hello, Okan!");

Console.Write("Enter your name: ");     // cursor stays on this line
string name = Console.ReadLine();
Console.Write("Enter your programme: ");
string programme = Console.ReadLine();

Console.WriteLine();                    // an empty line
Console.WriteLine("===== STUDENT CARD =====");
Console.WriteLine($"Name:\t{name}");    // \t = tab
```

```
11 Console.WriteLine($"Prog.:\t{programme}");
12 Console.WriteLine("Motto:\t\"Code, test, repeat.\"");  // \" = "
13 Console.WriteLine("Folder:\tC:\\MCMP113\\Week03");     // \\ = \
14 Console.WriteLine(@"Same:   C:\MCMP113\Week03");  // verbatim
15 Console.Write("Line 1\nLine 2\n");                 // \n = new line
```

Sample run

```
Hello, Okan!
Enter your name: Ayşe Yılmaz
Enter your programme: Computer Programming

===== STUDENT CARD =====
Name:   Ayşe Yılmaz
Prog.:  Computer Programming
Motto:  "Code, test, repeat."
Folder: C:\MCMP113\Week03
Same:   C:\MCMP113\Week03
Line 1
Line 2
```

The program stops at each `ReadLine` until the user presses Enter. The typed text is shown by the terminal, not printed by the program. If Turkish letters appear as `?` on your console, add `Console.OutputEncoding = System.Text.Encoding.UTF8;` as the first statement.

3.4 String interpolation

A string that starts with `$` may contain expressions in braces: `$"Name: {name}"` inserts the value of `name`. The older alternative is concatenation with `+` (`"Name: " + name`); interpolation is easier to read. Format specifiers inside the braces are described in Section 8.

>_ 4 Identifiers and variables

4.1 Naming rules

Names of variables, constants, methods and classes are called **identifiers**. The compiler enforces these rules:

1. letters, digits and the underscore _ only (no spaces, no `-`, `%`, `.`, ...);
2. the first character must not be a digit;
3. a name must not be a **reserved keyword** such as `int`, `double`, `class`, `if`, `for`, `while`, `string`, `const`, `new`, `return`;
4. upper and lower case are different (`total` ≠ `Total`).

Name	Legal?	Reason
`studentName`	yes	letters only (camelCase)
`total2`	yes	digits are allowed after the first character
`_count`	yes	underscore allowed, also at the start
`2ndPlace`	no	starts with a digit
`first name`	no	contains a space
`price-tl`	no	`-` is the minus operator
`class`	no	reserved keyword
`öğrenciAdı`	yes	Unicode letters are legal, but we use English names

4.2 Naming conventions

Conventions are not checked by the compiler but followed by professional C# programmers (Microsoft's coding conventions): **camelCase** for local variables (`examGrade`, `unitPrice`), **PascalCase** for constants, methods and types (`VatRate`, `WriteLine`, `Console`). Above all, a name should say *what the variable stores*: `examGrade` rather than `x`, `studentCount` rather than `n`.

4.3 Variables: a named box in memory

Definition – Variable

A **variable** is a named storage location in memory. It has a **name**, a **type** (which values fit into it and how many bytes it occupies) and a **value**, which can change while the program runs.

```
int age;            // declaration: create the box
age = 19;           // assignment: store a value in it
int year = 2026;    // declaration + initialisation in one step
age = age + 1;      // read the old value, add 1, store the result back
```

The = sign is the **assignment operator**: evaluate the right-hand side, then store the result in the variable on the left (pseudocode $age \leftarrow age + 1$). It is *not* a mathematical equation: $age = age + 1$ has no solution in mathematics, but in C# it simply increases `age`. Comparison for equality uses == (Weeks 4–5). The old value of a variable is lost when a new value is assigned.

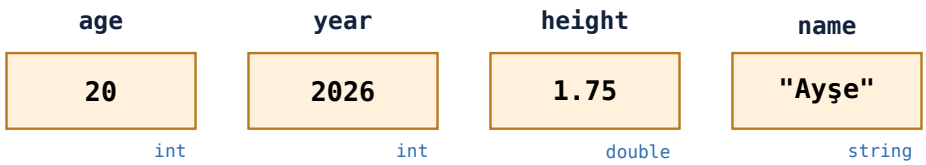


Figure 2: Four variables after the statements above (plus two more): name above the box, value inside, type below.

4.4 Trace tables

A **trace table** records the value of every variable after each statement — exactly as the computer executes them, top to bottom. Trace the four statements above and then this program:

```
1 int x = 5;
2 int y = x * 2;
3 x = x + 3;
4 y = y - x;
5 Console.WriteLine($"x = {x}, y = {y}");
```

Line	x	y	Output
1	5	–	
2	5	10	
3	8	10	
4	8	2	
5	8	2	`x = 8, y = 2`

4.5 Using a variable before it has a value

C# refuses to read a local variable that has not *definitely* been given a value:

```
int total;
Console.WriteLine(total);   // error CS0165: Use of unassigned local variable 'total'
```

Fix: initialise it, `int total = 0;`. (Some older languages would silently use whatever happened to be in memory.) Declaring the same name twice in one scope is also an error: `int x = 5; int x = 6;` gives CS0128. To change the value, write `x = 6;` without the type.

4.6 Example 8: swapping two variables

Swapping two values is a classic exercise. The obvious attempt `a = b; b = a;` fails: after `a = b` both variables contain the value of `b` and the old value of `a` is lost. As with two full glasses, we need a third, empty one.

Listing 3: `code/03_08_SwapVariables` – swap with a temporary variable

```
Console.Write("a = ");
int a = int.Parse(Console.ReadLine());
Console.Write("b = ");
int b = int.Parse(Console.ReadLine());
Console.WriteLine($"Before: a = {a}, b = {b}");

int temp = a;    // 1) save a copy of a
a = b;           // 2) overwrite a with b
b = temp;        // 3) b gets the saved copy

Console.WriteLine($"After : a = {a}, b = {b}");
```

Sample run

```
a = 5
b = 9
Before: a = 5, b = 9
After : a = 9, b = 5
```

Statement	a	b	temp
(input)	5	9	–
`int temp = a;`	5	9	5
`a = b;`	9	9	5
`b = temp;`	9	5	5

>_ 5 Data types

5.1 The built-in types

Every variable has a **data type**. The type determines which values can be stored, how much memory is used and which operations are allowed. The table lists the types we use; sizes and ranges were printed by Example 10 below.

Type	Bytes	Range / precision	Literal	Typical use
`byte`	1	0 . . . 255	`200`	small counts, raw data
`short`	2	−32 768 . . . 32 767	`30000`	rarely needed
`int`	4	−2 147 483 648 . . . 2 147 483 647	`42`	**default whole number**
`long`	8	$\approx \pm 9.2 \times 10^{18}$	`42L`	very large whole numbers
`float`	4	$\approx \pm 3.4 \times 10^{38}$, 6–9 significant digits	`3.14f`	graphics, sensors
`double`	8	$\approx \pm 1.7 \times 10^{308}$, 15–17 significant digits	`3.14`	**default real number**
`decimal`	16	$\approx \pm 7.9 \times 10^{28}$, 28–29 significant digits	`19.99m`	**money**
`char`	2	one UTF-16 character, U+0000 ...U+FFFF	`'A'`	a letter, a grade
`bool`	1	`true` or `false`	`true`	yes/no, passed?
`string`	varies	any text (a sequence of `char`)	`"Ayşe"`	names, IDs, phone numbers

Literals and suffixes. A whole-number literal such as `42` has type `int`; a literal with a decimal point or exponent such as `3.14` or `2.5e3` (= 2500) has type `double`. For other types add a suffix:

`L` for `long`, `f` for `float`, `m` for `decimal`. Without the suffix, `float f = 3.14;` and `decimal price = 19.99;` do not compile (CS0664: *Literal of type double cannot be implicitly converted to type 'decimal'; use an 'M' suffix*). For `byte` and `short` the constant is simply checked: `byte b = 200;` is fine, `byte b = 256;` gives CS0031. Long numbers may be grouped with underscores: `1_000_000`.

`char`, `string` and `bool`. A `char` literal uses single quotes (`'A'`), a `string` literal double quotes (`"A"`). Internally a `char` is a number (`'A'` is 65). A `bool` holds the result of a yes/no question and becomes important with decisions in Week 5. `string` is different from the other types in the table: it is a *reference type* whose size depends on the length of the text.

5.2 Real numbers: `float`, `double` or `decimal`?

`float` and `double` store numbers in *binary*. Most decimal fractions, such as 0.1, have no exact binary representation (just as $1/3 = 0.333\ldots$ has none in decimal), so tiny rounding errors appear. `decimal` stores numbers in base 10 and represents amounts such as 0.1 or 19.99 exactly — that is why money is stored in `decimal`. Real output (.NET 10):

```
Console.WriteLine(0.1 + 0.2);    // double
Console.WriteLine(0.1f + 0.2f);  // float
Console.WriteLine(0.1m + 0.2m);  // decimal
Console.WriteLine(1.0 / 3);
Console.WriteLine(1.0f / 3);
Console.WriteLine(1.0m / 3);
```

Output

```
0.30000000000000004
0.3
0.3
0.3333333333333333
0.33333334
0.3333333333333333333333333333
```

The `float` sum prints `0.3` only because `float` shows fewer digits; its value is not exact either. The last three lines show the number of significant digits kept: about 7 for `float`, 16 for `double`, 28 for `decimal`. `decimal` is slower and has a much smaller range than `double`, so for measurements, science and engineering we use `double`.

5.3 Choosing a type

Use `int` for whole numbers such as counts, ages and exam points (even when `byte` would suffice, `int` is the default); `long` when values can exceed $\approx$ 2.1 billion (world population $\approx$ 8.2 billion); `double` for measured quantities (temperature, height, distance); `decimal` for money; `bool` for yes/no facts ("passed?"); `char` for a single character (`'B'`); and `string` for names, T.C. Kimlik numbers and phone numbers — we never calculate with them, a phone number may start with 0 or contain `+`, and an 11-digit T.C. number would not even fit in an `int`.

Rule of thumb

If you never do arithmetic with it, it is not a number: store identifiers, codes and phone numbers as `string`.

5.4 Example 10: asking C# for the limits; `var`

`sizeof(T)` gives the size of a type in bytes; `T.MinValue` and `T.MaxValue` give its range. The second part of the program uses `var`: the compiler *infers* the type from the value on the right-hand side. The variable is still strongly typed — `var a = 42;` makes `a` an `int` forever — and it must be initialised (`var x;` gives CS0818). `GetType().Name` prints the .NET name of the type (`Int32` = `int`, `Int64` = `long`, `Single` = `float`, `Double` = `double`, `Decimal` = `decimal`). While learning, write the type explicitly; `var` is fine when the type is obvious.

Listing 4: `code/03_10_TypeLimits` – sizes and ranges; var (lines 7–10, the same pattern for long, float, double and decimal, are omitted)

```
1  Console.WriteLine($"{"type",-8}{"bytes",6}  {"MinValue",-32}{"MaxValue"}");
2  Console.WriteLine($"{"byte",-8}{sizeof(byte),6}  {byte.MinValue,-32}{byte.MaxValue}");
3  Console.WriteLine($"{"short",-8}{sizeof(short),6}  {short.MinValue,-32}{short.MaxValue}");
4  Console.WriteLine($"{"int",-8}{sizeof(int),6}  {int.MinValue,-32}{int.MaxValue}");
5  Console.WriteLine($"{"char",-8}{sizeof(char),6}  {(int)char.MinValue,-32}{(int)char.
       MaxValue}");
6  Console.WriteLine($"{"bool",-8}{sizeof(bool),6}  {false,-32}{true}");
7
8  // var: the compiler infers the type from the literal on the right-hand side
9  var a = 42;
10 var b = 42L;
11 var c = 4.2;
12 var d = 4.2f;
13 var e = 4.2m;
14 var f = 'x';
15 var g = "x";
16 Console.WriteLine();
17 Console.WriteLine($"42 -> {a.GetType().Name}, 42L -> {b.GetType().Name}, 4.2 -> {c.
       GetType().Name}");
18 Console.WriteLine($"4.2f -> {d.GetType().Name}, 4.2m -> {e.GetType().Name}, 'x' -> {f.
       GetType().Name}, \"x\" -> {g.GetType().Name}");
```

Output

```
type     bytes  MinValue                          MaxValue
byte         1  0                                 255
short        2  -32768                           32767
int          4  -2147483648                      2147483647
long         8  -9223372036854775808             9223372036854775807
float        4  -3.4028235E+38                   3.4028235E+38
double       8  -1.7976931348623157E+308         1.7976931348623157E+308
decimal     16  -79228162514264337593543950335   79228162514264337593543950335
char         2  0                                 65535
bool         1  False                             True

42 -> Int32, 42L -> Int64, 4.2 -> Double
4.2f -> Single, 4.2m -> Decimal, 'x' -> Char, "x" -> String
```

Note the notation `3.4028235E+38`: `E+38` means $\times 10^{38}$.

>_ 6 Constants

A **constant** is a named value that can never change. It is declared with `const` and must be initialised in the declaration:

```
const double Pi = 3.14159;
const decimal VatRate = 0.20m;
Pi = 3.0;     // error CS0131: The left-hand side of an assignment must be a variable, ...
```

Why use constants? (1) The name documents the meaning — `VatRate` is clearer than a "magic number" `0.20m` in a formula; (2) if the value changes (a new VAT rate), you change one line; (3) the compiler prevents accidental changes. Constants are named in PascalCase. .NET already provides important mathematical constants, e.g. `Math.PI` $= 3.141592653589793$.

6.1 Example 3: circle area and circumference

For a circle of radius r: area $A = \pi r^2$, circumference $C = 2\pi r$.

Listing 5: `code/03_03_CircleArea` – const and Math.PI

```
1  const double Pi = 3.14159;      // our constant: can never change
2
3  Console.Write("Radius (cm): ");
4  double radius = double.Parse(Console.ReadLine());
5
6  double area = Pi * radius * radius;          // A = πr²
7  double circumference = 2 * Pi * radius;      // C = π2r
8  double areaExact = Math.PI * radius * radius;
9
10 Console.WriteLine($"Area          : {area:F4} cm²");
11 Console.WriteLine($"Circumference : {circumference:F4} cm");
12 Console.WriteLine($"Area (Math.PI): {areaExact:F4} cm²");
```

Sample run

```
Radius (cm): 2.5
Area          : 19.6349 cm²
Circumference : 15.7080 cm
Area (Math.PI): 19.6350 cm²
```

Check: $\pi \cdot 2.5^2 = 19.63495\ldots$; with 3.14159 we get 19.6349375, so the fourth decimal differs. Use `Math.PI` in real programs.

6.2 Example 7: a receipt with `decimal` and KDV

Problem. Read a product name, its net unit price and a quantity; print a receipt with subtotal, KDV (value-added tax, 20 %) and total.

IPO table

Input	`product`, `unitPrice`, `quantity`
Process	subtotal, VAT, total
Output	aligned receipt

Algorithm – Receipt

1 **START**
2 **READ** *product*, *unitPrice*, *quantity*
3 *subtotal* $\leftarrow$ *unitPrice* $\times$ *quantity*
4 *vat* $\leftarrow$ *subtotal* $\times$ 0.20
5 *total* $\leftarrow$ *subtotal* $+$ *vat*
6 **PRINT** *subtotal*, *vat*, *total*
7 **END**

Listing 6: `code/03_07_Receipt` – decimal, const and aligned output

```
const decimal VatRate = 0.20m;      // KDV 20 %  (m = decimal)

Console.Write("Product name        : ");
string product = Console.ReadLine();
Console.Write("Unit price (TL, net): ");
decimal unitPrice = decimal.Parse(Console.ReadLine());
Console.Write("Quantity            : ");
int quantity = int.Parse(Console.ReadLine());

decimal subtotal = unitPrice * quantity;  // int -> decimal
decimal vat = subtotal * VatRate;
decimal total = subtotal + vat;

Console.WriteLine("--------------------------------");
Console.WriteLine($"{product,-12} {quantity,3} x {unitPrice,8:F2}");
Console.WriteLine($"{"Subtotal",-18} {subtotal,10:F2} TL");
Console.WriteLine($"{"KDV 20%",-18} {vat,10:F2} TL");
Console.WriteLine($"{"TOTAL",-18} {total,10:F2} TL");
```

Sample run

```
Product name        : Notebook
Unit price (TL, net): 24.90
Quantity            : 3
--------------------------------
Notebook       3 x    24.90
Subtotal                74.70 TL
KDV 20%                 14.94 TL
TOTAL                   89.64 TL
```

Check by hand: $24.90 \times 3 = 74.70$; $74.70 \times 0.20 = 14.94$; $74.70 + 14.94 = 89.64$. The `int quantity` is converted to `decimal` automatically in `unitPrice * quantity`. (In Turkish shops, prices usually *include* KDV; here the price is net, so KDV is added.)

>_ 7 Input and type conversion

7.1 `Console.ReadLine()` returns a string

Whatever the user types — even `19` — arrives as *text*. `int age = Console.ReadLine();` does not compile (CS0029: *Cannot implicitly convert type 'string' to 'int'*). To calculate, the text must be **parsed** into a number:

$$\texttt{"19"}\ \text{(string)} \xrightarrow{\texttt{int.Parse}} \texttt{19}\ \text{(int)}$$

Listing 7: `code/03_12_InputParsing` – ReadLine, Parse, Convert and a preview of TryParse

```
Console.Write("Your age: ");
string ageText = Console.ReadLine();   // text, e.g. "19"
int age = int.Parse(ageText);          // number 19

Console.WriteLine($"Text   + 1 = {ageText + 1}");  // joins text
Console.WriteLine($"Number + 1 = {age + 1}");      // adds

Console.Write("Height in m (e.g. 1.75): ");
double height = double.Parse(Console.ReadLine());
Console.WriteLine($"Height in cm: {height * 100}");

Console.Write("Credits this term: ");
```

```
13 int credits = Convert.ToInt32(Console.ReadLine());
14 Console.WriteLine($"Credits: {credits}");
15
16 // Preview (Week 5): TryParse never crashes; it returns a bool.
17 Console.Write("Lucky number: ");
18 bool ok = int.TryParse(Console.ReadLine(), out int lucky);
19 Console.WriteLine($"Success: {ok}, value: {lucky}");
```

Sample run

```
Your age: 19
Text   + 1 = 191
Number + 1 = 20
Height in m (e.g. 1.75): 1.75
Height in cm: 175
Credits this term: 30
Credits: 30
Lucky number: 7
Success: True, value: 7
```

Line 7 shows a classic surprise: `ageText + 1` is `"191"`, because `+` with a string *joins* text (**concatenation**). With the number `age` it adds.

7.2 `Parse`, `Convert` and `TryParse`

Text	int.Parse(text)	Convert.ToInt32(text)	int.TryParse(text, out int n)
`"42"`	`42`	`42`	`true, n = 42`
`" 42 "`	`42`	`42`	`true, n = 42`
`"3.5"`	`FormatException`	`FormatException`	`false, n = 0`
`"abc", ""`	`FormatException`	`FormatException`	`false, n = 0`
`"99999999999"`	`OverflowException`	`OverflowException`	`false, n = 0`
`null`	`ArgumentNullException`	`0`	`false, n = 0`

`double.Parse`, `decimal.Parse` and `long.Parse` work in the same way. An *exception* that is not handled stops the program. For example, typing `nineteen` in Example 12 gives (stack trace abridged):

Crash

```
Your age: nineteen
Unhandled exception. System.FormatException: The input string 'nineteen' was not in a
    correct format.
   at System.Int32.Parse(String s)
   at Program.<Main>$(String[] args) in ...\03_12_InputParsing\Program.cs:line 5
```

`TryParse` never crashes: it returns `true` or `false` and delivers the number through an `out` variable (0 if the conversion failed). With the input `seven` for the last question, Example 12 prints `Success: False, value: 0`. We will combine `TryParse` with `if` in Week 5.

7.3 Culture: decimal point or decimal comma?

In Turkey the decimal separator is a comma (72,5) and the thousands separator is a point (1.234.567). `double.Parse` uses the computer's **culture** (regional settings) unless told otherwise. We tested both situations with .NET 10:

Typed	Turkish PC, default project	Our template (invariant culture)
`72.5`	725 — silently wrong	72.5
`72,5`	72.5	725 — silently wrong
printing 72.5	`72,5`	`72.5`

The wrong separator is interpreted as a *thousands* separator, so there is *no error message* — the program just calculates with a number 10 times too large. Real run of Example 6 with a comma:

Sample run – wrong input

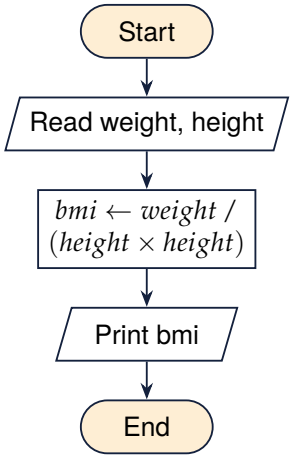
```
Weight (kg): 72,5
Height (m) : 1.78
BMI = 228.8 kg/m²
```

Course rule

Our template sets `InvariantGlobalization=true`, so every computer uses the decimal **point**. Always type `72.5`, `1.78`, `24.90`. In a project without our template, you can fix one conversion explicitly with `using System.Globalization;` and `double.Parse(text, CultureInfo.InvariantCulture)`.

7.4 Example 6: Body Mass Index

$\text{BMI} = \dfrac{\text{weight}}{\text{height}^2}$ with weight in kg and height in m.



Listing 8: `code/03_06_Bmi` – a sequence program

```
Console.Write("Weight (kg): ");
double weight = double.Parse(Console.ReadLine());
Console.Write("Height (m) : ");
double height = double.Parse(Console.ReadLine());

// parentheses: divide by the SQUARE of the height
double bmi = weight / (height * height);

Console.WriteLine($"BMI = {bmi:F1} kg/m²");
```

Sample run

```
Weight (kg): 72.5
Height (m) : 1.78
BMI = 22.9 kg/m²
```

Check: $1.78^2 = 3.1684$ and $72.5/3.1684 = 22.88\ldots$, printed with one decimal as 22.9. The parentheses are essential: `weight / height * height` divides and then multiplies again, giving back 72.5. The BMI categories need decisions (Week 5).

7.5 Example 5: Celsius to Fahrenheit

$F = \frac{9}{5}C + 32$ — exercise F of Week 1, now in C#.

Listing 9: `code/03_05_CelsiusToFahrenheit` – a formula with double

```
Console.Write("Temperature in °C: ");
double celsius = double.Parse(Console.ReadLine());

// celsius is a double, so celsius * 9 / 5 is computed as double
double fahrenheit = celsius * 9 / 5 + 32;

Console.WriteLine($"{celsius} °C = {fahrenheit:F1} °F");
```

Three runs

```
Temperature in °C: 36.6
36.6 °C = 97.9 °F
Temperature in °C: 100
100 °C = 212.0 °F
Temperature in °C: -40
-40 °C = -40.0 °F
```

The test values follow the Week 1 advice: a normal value (36.6 → 97.88), a known value (100 → 212) and a special point (-40, where both scales agree).

Integer division

`9 / 5` with two `int` literals is *integer division* and gives `1`, not `1.8`. Therefore `9 / 5 * c + 32` computes `c + 32`; for `c = 20` it prints `52` instead of `68` (verified). Our program avoids the problem because the expression is evaluated from left to right and `celsius * 9` is already a `double`. The safest style is `9.0 / 5`. Week 4 covers division in detail.

7.6 Example 4: rectangle

Listing 10: `code/03_04_Rectangle` – input – process – output

```
Console.Write("Width  (m): ");
double width = double.Parse(Console.ReadLine());
Console.Write("Height (m): ");
double height = double.Parse(Console.ReadLine());

double area = width * height;
double perimeter = 2 * (width + height);

Console.WriteLine($"Area      = {area} m²");
Console.WriteLine($"Perimeter = {perimeter} m");
```

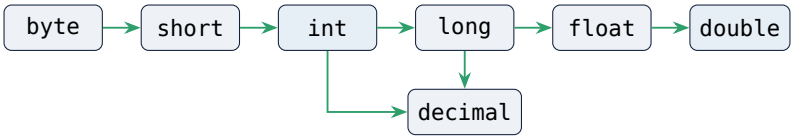
Sample run

```
Width  (m): 4.5
Height (m): 3
Area      = 13.5 m²
Perimeter = 15 m
```

Without a format specifier a `double` is printed with as many digits as needed: `15`, not `15.0`.

7.7 Implicit and explicit conversions

C# converts a value *implicitly* (automatically) when the target type can hold every value of the source type — a **widening** conversion:



Conversions in the opposite direction may lose information and need an explicit **cast** `(type)value`; without it the compiler reports CS0266 (*Cannot implicitly convert type 'double' to 'int'. An explicit conversion exists (are you missing a cast?)*). Between `decimal` and `float`/`double` a cast is needed in both directions. Note that `int`→`float` and `long`→`float`/`double` are implicit but may lose digits of very large numbers: the `int` 123 456 789 becomes the `float` 123 456 792.

Listing 11: `code/03_11_CastDemo` – implicit and explicit conversions

```
int count = 7;
long bigCount = count;     // implicit: int -> long always fits
double ratio = count;      // implicit: int -> double
Console.WriteLine($"int {count} -> long {bigCount} -> double {ratio}");

double price = 3.99;
int truncated = (int)price;            // cast: fraction CUT OFF
int rounded = Convert.ToInt32(price);  // Convert: rounds
Console.WriteLine($"(int){price} = {truncated}");
Console.WriteLine($"Convert.ToInt32({price}) = {rounded}");

double negative = -3.99;
Console.WriteLine($"(int){negative} = {(int)negative}");

double precise = 1.0 / 3;
float lessPrecise = (float)precise;    // fewer digits kept
Console.WriteLine($"double {precise} -> float {lessPrecise}");

```

```
int big = 300;
byte small = (byte)big;    // 300 does not fit in 0..255
Console.WriteLine($"(byte){big} = {small}");

int max = int.MaxValue;
int wrapped = max + 1;     // no error: wraps to the minimum!
Console.WriteLine($"{max} + 1 = {wrapped}");
```

Output

```
int 7 -> long 7 -> double 7
(int)3.99 = 3
Convert.ToInt32(3.99) = 4
(int)-3.99 = -3
double 0.3333333333333333 -> float 0.33333334
(byte)300 = 44
2147483647 + 1 = -2147483648
```

- A cast to an integer type **truncates** towards zero: $(int)3.99 = 3$ and $(int)(-3.99) = -3$.
- `Convert.ToInt32` *rounds*. At exactly .5 it rounds to the even neighbour: $6.5 \to 6$ but $7.5 \to 8$ ("banker's rounding", discussed in Week 4).
- `(byte)300`: a `byte` keeps only the lowest 8 bits, $300 - 256 = 44$.
- `int.MaxValue + 1` *wraps around* to `int.MinValue` — see the next subsection.

7.8 Overflow and `checked`

An `int` has 32 bits and cannot represent values above $2^{31} - 1 = 2\,147\,483\,647$. By default C# does not report an overflow at run time; the result wraps around like a car's odometer. In Example 13, $2\,000\,000\,000 + 500\,000\,000 = 2\,500\,000\,000$, which exceeds `int.MaxValue`; the stored result is $2\,500\,000\,000 - 2^{32} = 2\,500\,000\,000 - 4\,294\,967\,296 = -1\,794\,967\,296$. There are three remedies: use a larger type (`long`), wrap the expression in `checked(…)` so that an `OverflowException` stops the program instead of producing garbage, or check the input.

Listing 12: `code/03_13_OverflowChecked` – silent overflow vs. checked

```
Console.Write("Start value: ");
int start = int.Parse(Console.ReadLine());
Console.Write("Add        : ");
int step = int.Parse(Console.ReadLine());

int silent = start + step;             // may wrap around
Console.WriteLine($"int     : {silent}");

long exact = (long)start + step;       // long has room
Console.WriteLine($"long    : {exact}");

int safe = checked(start + step);      // throws on overflow
Console.WriteLine($"checked : {safe}");
```

Run 1

```
Start value: 100
Add        : 23
int      : 123
long     : 123
checked : 123
```

Run 2 (stack trace abridged)

```
Start value: 2000000000
Add        : 500000000
int      : -1794967296
long     : 2500000000

Unhandled exception. System.OverflowException:
    Arithmetic operation resulted in an overflow.
```

With constants, the compiler detects overflow already: `int big = int.MaxValue + 1;` gives CS0220 *The operation overflows at compile time in checked mode*.

>_ 8 Formatted output

Inside an interpolated string, `{value:format}` chooses a **format specifier** and `{value,width}` a field width (positive = right-aligned, negative = left-aligned). Both can be combined: `{subtotal,10:F2}`. All results below were printed with our (invariant) template:

In code	Value	Printed	Meaning
`{x}`	`1.0 / 3`	`0.3333333333333333`	default: shortest exact representation
`{x:F2}`	`3.14159`	`3.14`	fixed point, 2 decimals (rounded for display)
`{n:N0}`	`1234567`	`1,234,567`	thousands separators, 0 decimals
`{x:N2}`	`1234.5`	`1,234.50`	separators and 2 decimals
`{h:D2}`	`7`	`07`	integer with at least 2 digits
`[{n,5}]`	`42`	`[   42]`	width 5, right-aligned
`[{n,-5}]`	`42`	`[42   ]`	width 5, left-aligned
`{p:P0}`	`0.2`	`20 %`	percent
`{d:C}`	`1234.5m`	`¤1,234.50`	currency: invariant sign ¤ — avoid

A format specifier changes only how the value is *shown*; the variable keeps all its digits. Because the invariant culture has no real currency, `C` prints the generic sign ¤ (a Turkish PC would print `₺1.234,50`); in this course we print money as `$"{total:F2} TL"`.

8.1 Example 9: seconds to hours, minutes and seconds

With two `int` operands, `/` gives the whole-number quotient and `%` the remainder (a preview of Week 4).

Algorithm – Seconds to h:m:s

1 **START**
2 **READ** *total*
3 $h \leftarrow \lfloor total / 3600 \rfloor$
4 $m \leftarrow \lfloor (total \bmod 3600) / 60 \rfloor$
5 $s \leftarrow total \bmod 60$
6 **PRINT** h, m, s
7 **END**

Expression	$total = 3725$
`total / 3600`	1
`total % 3600`	125
`125 / 60`	2
`total % 60`	5

Listing 13: `code/03_09_SecondsToHms` – integer division, remainder and D2

```
Console.Write("Total seconds: ");
int totalSeconds = int.Parse(Console.ReadLine());

int hours = totalSeconds / 3600;         // whole hours
int minutes = totalSeconds % 3600 / 60;  // leftover minutes
int seconds = totalSeconds % 60;         // leftover seconds

Console.WriteLine($"{hours} h {minutes} min {seconds} s");
Console.WriteLine($"Clock: {hours:D2}:{minutes:D2}:{seconds:D2}");
```

Two runs

```
Total seconds: 3725
1 h 2 min 5 s
Clock: 01:02:05
Total seconds: 86399
23 h 59 min 59 s
Clock: 23:59:59
```

>_ 9 Common mistakes

9.1 Compiler errors

The messages below are the real messages of the .NET 10 compiler. Read the *first* error first: it gives file, line and column, e.g. `Program.cs(3,17)`. A syntax error such as a missing semicolon hides the other errors until it is fixed, so "fix, rebuild, repeat".

You wrote	Compiler says	Fix
`int age = Console.ReadLine();`	CS0029: Cannot implicitly convert type 'string' to 'int'	`int.Parse(Console.ReadLine())`
`decimal price = 19.99;`	CS0664: Literal of type double cannot be implicitly converted to type 'decimal'; use an 'M' suffix ...	`19.99m`
`float f = 3.14;`	CS0664: ...use an 'F' suffix ...	`3.14f`
`int n = 3.9;`	CS0266: Cannot implicitly convert type 'double' to 'int'. An explicit conversion exists ...	`(int)3.9` (if 3 is wanted)
`int total; …(total)`	CS0165: Use of unassigned local variable 'total'	`int total = 0;`
`console.WriteLine(x);`	CS0103: The name 'console' does not exist in the current context	`Console`
`Console.WriteLine(x)`	CS1002: ; expected	`add ;`
`Pi = 3.0;` (`Pi` is const)	CS0131: The left-hand side of an assignment must be a variable, property or indexer	do not change constants
`int x = 5; int x = 6;`	CS0128: A local variable or function named 'x' is already defined in this scope	`x = 6;`
`"C:\Temp\new"`	CS1009: Unrecognized escape sequence	`@"C:\Temp\new"`

9.2 Silent bugs

It compiles, but the answer is wrong

These are more dangerous than compiler errors, because nothing warns you. Only tests with known answers reveal them (all verified with real runs):

- **Text plus number joins:** `"Total: " + 2 + 3` prints `Total: 23`. Use `"Total: " + (2 + 3)`.
- **Integer division:** `9 / 5` is 1; `9 / 5 * 20 + 32` prints 52, not 68. Use `9.0 / 5`.

- **Comma instead of point:** with our template `double.Parse("72,5")` returns `725`.
- **Overflow:** an `int` variable holding `int.MaxValue` plus 1 gives `-2147483648`. Use `long` or `checked`.
- **Forgotten parentheses:** `weight / height * height` is not $weight/height^2$; `g1 + g2 + g3 / 3` is not the average (Week 1).
- **= versus ==:** = assigns, == compares. In C#, `if (n = 5)` does not compile (CS0029: *Cannot implicitly convert type 'int' to 'bool'*), but the confusion causes wrong reasoning in pseudocode and trace tables.

>_ 10 Summary

1. C# is compiled to IL (`dotnet build`); the .NET runtime JIT-compiles IL to machine code when the program runs. Developers install the SDK, users need only the runtime.
2. A console project consists of `Program.cs` and a `.csproj` file; we use top-level statements and the course template (Nullable off, invariant globalization).
3. Statements end with `;`, C# is case-sensitive, comments are ignored by the compiler.
4. A variable has a name, a type and a value; = stores a value (←); reading an unassigned variable is error CS0165.
5. Default types: `int` for whole numbers, `double` for measurements, `decimal` for money, `string` for text and identifiers, `bool` for yes/no, `char` for one character. Suffixes: `L`, `f`, `m`.
6. `const` protects fixed values; `Math.PI` is built in.
7. `Console.ReadLine()` returns a string: convert with `int.Parse`, `double.Parse`, `decimal.Parse` or `Convert`; type a decimal *point*.
8. Widening conversions are implicit; narrowing conversions need a cast, which truncates. `int` arithmetic can overflow silently; `checked` makes it an error.
9. Format output with `$"{x:F2}"`, `N0`, `D2` and field widths such as `{x,10:F2}`.

Glossary (English – Turkish)

programming language	programlama dili	data type	veri tipi
machine code	makine kodu	integer	tam sayı
assembly language	assembly dili	floating-point number	kayan noktalı sayı
high-level language	yüksek seviyeli dil	character / string	karakter / karakter dizisi
compiler / interpreter	derleyici / yorumlayıcı	Boolean (true/false)	mantıksal (doğru/yanlış)
runtime	çalışma zamanı	constant	sabit
statement	deyim / komut	literal	sabit değer (literal)
semicolon	noktalı virgül	type conversion / cast	tip dönüşümü / tür zorlama
comment	yorum satırı	implicit / explicit	örtük / açık
identifier	tanımlayıcı	overflow	taşma
keyword	anahtar sözcük	truncation	kesme (ondalığı atma)
variable	değişken	exception	istisna (çalışma hatası)
declaration	tanımlama / bildirim	escape sequence	kaçış dizisi
initialisation	ilk değer atama	string interpolation	metin enterpolasyonu
assignment	atama	format specifier	biçim belirteci
case-sensitive	büyük/küçük harfe duyarlı	decimal separator	ondalık ayırıcı

>_ 11 Exercises

Write every program as a separate project with our template. Each program starts with a header comment, uses meaningful names, prompts with `Console.Write` and is tested with at least two inputs whose correct results you computed by hand. Difficulty: ⋆ basic, ⋆⋆ intermediate, ⋆⋆⋆ challenging. Solutions (instructor edition) were compiled and run with .NET 10.

11.1 Quizzes from the lecture

Q1. ⋆ **Predict the output** of each `WriteLine`:

```
int a = 7;
int b = a;
a = 10;
Console.WriteLine($"{a} {b}");   // (1)
Console.WriteLine(5 / 2);          // (2)
Console.WriteLine(5 / 2.0);        // (3)
Console.WriteLine("5" + 2);        // (4)
Console.WriteLine(1 + 2 + "3");    // (5)
Console.WriteLine((int)9.99);      // (6)
```

Q2. ⋆⋆ **Find the bugs.** The program should read a price per kg, buy 2.5 kg, add 20 % KDV and print the total. Find all errors and correct the program.

```
Console.Write("Price per kg: ")
decimal price = Console.ReadLine();
const decimal Vat = 0.20;
int quantity = 2.5;
decimal Total = price * quantity * (1 + vat);
console.WriteLine(total);
```

11.2 In-class exercises

E1. ⋆ **Choose types** for: number of pages of a book, fuel price per litre, "is a member?", room code `"B12"`, distance in km, the initial letter of your name.

E2. ⋆ **Age calculator.** Read a name and a birth year; print e.g. "Ayşe, in 2026 you turn 19.". Use `const int CurrentYear = 2026;`.

E3. ⋆ **Kilometres to miles.** Read a distance in km and print it in miles with 3 decimals; 1 mile = 1.609344 km (a constant).

E4. ⋆ **Average of three grades.** Read three grades as `double` and print the average with two decimals.

E5. ⋆⋆ **Back to seconds.** The reverse of Example 9: read hours, minutes and seconds and print the total number of seconds.

E6. ⋆⋆ **Swap trace.** Trace `t = x; x = y; y = t;` for $x = 3$, $y = 8$. Then trace the version without a temporary variable: `x = x + y; y = x - y; x = x - y;`.

11.3 Homework (before Week 4)

H1. ⋆ **Cylinder volume.** Read the radius r and height h (cm); print $V = \pi r^2 h$ with two decimals (use `Math.PI`).

H2. ⋆ **Currency converter.** Read an amount in TL and the exchange rate (TL per 1 EUR) typed by the user; print the amount in EUR with two decimals. Which type do you use and why?

H3. ⋆⋆ **Minutes to days.** Read a number of minutes and print it as days, hours and minutes.

H4. ⋆⋆ **Short overflow.** Predict, then run: `short s = short.MaxValue; s = (short)(s + 1);`. Why is the cast needed? Do the same with `byte`.

H5. Read Sections 2–9 again and solve three problems from the extra practice.

11.4 Extra practice

P1. ⋆ Which declarations compile? (a) `int n = 12;` (b) `long big = 5000000000;` (c) `float f = 1.5;` (d) `char c = "A";` (e) `bool ok = true;` (f) `string code = 104;` (g) `decimal d = 12;`

P2. ⋆ Identify the legal identifiers: `total_sum`, `3rdGrade`, `if`, `myAge`, `my-age`, `_x1`, `Class`.

P3. ⋆⋆ **Fuel cost.** Read the distance (km), the consumption (litres per 100 km) and the fuel price (TL per litre); print the litres needed and the cost.

P4. ⋆⋆ **Banknotes.** Read a whole amount in TL and print how many 100, 50, 20, 10, 5 and 1 TL pieces are needed (use `/` and `%`).

P5. ⋆⋆ **Trace.** Trace `int x = 10; int y = 3; int z = x / y; x = x % y; y = z * y + x;` and give the final values.

P6. ⋆ ⋆ ⋆ **Explain the output** (our template) of: `(byte)big` with `int big = 260;`, `(int)-7.8`, `Convert.ToInt32(6.5)`, `Convert.ToInt32(7.5)`, `double.Parse("1,500")`, `7 / 2 * 2.0`, `7 / 2.0 * 2`.

P7. ⋆ ⋆ ⋆ **Seconds to days.** Read a (possibly very large) number of seconds and print it as `d hh:mm:ss`. Which type allows inputs above 2 147 483 647?

P8. ⋆ ⋆ ⋆ **Research.** Find out why `0.1 + 0.2` is not exactly `0.3` for a `double`. Write 0.1 as a binary fraction (first 10 bits after the point).

References and further reading

[1] Microsoft Learn, *Built-in types* (*C# reference*). https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/built-in-types

[2] Microsoft Learn, *Integral numeric types*. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/integral-numeric-types

[3] Microsoft Learn, *Floating-point numeric types*. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/floating-point-numeric-types

[4] Microsoft Learn, *Built-in numeric conversions*. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/numeric-conversions

[5] Microsoft Learn, *checked and unchecked statements*. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/checked-and-unchecked

[6] Microsoft Learn, *String interpolation using $*. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/tokens/interpolated

[7] Microsoft Learn, *Standard numeric format strings*. https://learn.microsoft.com/en-us/dotnet/standard/base-types/standard-numeric-format-strings

[8] Microsoft Learn, *Top-level statements*. https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/program-structure/top-level-statements

[9] Microsoft Learn, *C# identifier naming rules and conventions*. https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/identifier-names

[10] Microsoft Learn, *.NET globalization and ICU* (*invariant mode*). https://learn.microsoft.com/en-us/dotnet/core/runtime-config/globalization

[11] Microsoft Learn, *Managed execution process* (IL and JIT). https://learn.microsoft.com/en-us/dotnet/standard/managed-execution-process